



P

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043
Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026-27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 10

Team No.: 8

Project Title: Multithreaded Linux Application Using POSIX Threads and Mutexes

Team Members

Roll Number	Student Name	Signature
2520090170	S Svara Haasini	<i>S. Haasini</i>
2520090227	B Sudharshini	<i>Sudharshini</i>
2520090190	Sumanjali	<i>Sumanjali</i>

1. Abstract

(Summarize the project idea, problem background, relevant Operating Systems/System Programming concepts, proposed Linux-based solution, and expected outcome. The abstract should not exceed 250 words.)

Modern multi-core systems rely on concurrent execution for efficiency and responsiveness, but when multiple threads within a process access shared resources at the same time, it can cause race conditions and inconsistent results. This is a key challenge in Operating Systems, since threads share the same memory space and, without proper synchronization, can corrupt shared data. This project addresses this by developing a multithreaded Linux application using POSIX threads (Pthreads) for concurrent execution and mutex locks to synchronize access to critical sections, ensuring correctness and thread safety.

2. Problem Statement

(Clearly describe the problem that the project intends to address and its relevance to Operating Systems and Systems Programming.)

Modern applications increasingly rely on concurrent execution to make efficient use of multi-core processors and to remain responsive while performing multiple operations at once. However, when several threads within the same process access shared resources simultaneously, it can lead to race conditions, data inconsistency, and unpredictable program behaviour. This is a fundamental challenge in Operating Systems and Systems Programming, since the OS provides threads as lightweight units of execution that share the same address space, memory, and file descriptors of their parent process. Without proper synchronization mechanisms such as mutexes, concurrent access to shared data structures can corrupt results and cause the application to behave incorrectly or crash. This project addresses the problem of safely coordinating multiple threads that access shared resources by designing and implementing a

multithreaded Linux application that uses POSIX threads (Pthreads) for concurrent execution and mutex locks for synchronizing access to critical sections, thereby ensuring correctness, data consistency, and thread safety.

3. Objectives

(List the specific objectives of the project.)

1. To understand and apply the concept of multithreading in a Linux environment using the POSIX Threads (Pthreads) library.
2. To design and implement a C program that creates multiple threads to perform concurrent tasks efficiently.
3. To identify critical sections in shared data access and use mutex locks to prevent race conditions and ensure data consistency.
4. To evaluate thread synchronization, thread joining, and safe resource sharing, and to demonstrate correct, thread-safe program execution on Linux.

4. Proposed Methodology

(Briefly explain how the project will be designed and implemented in Linux. Describe the major functionality, approach, and flow of the proposed solution.)

The project will be developed in the C programming language on a Linux (Ubuntu) platform using the GCC compiler. The main program will create a fixed number of worker threads using `pthread_create()`, each of which will execute a common thread function to perform a task such as updating a shared counter or shared data structure. Since all threads operate within the same process address space, a global shared resource (for example, a shared counter, shared array, or a simulated bank balance) will be accessed by multiple threads concurrently.

To prevent race conditions, a mutex (`pthread_mutex_t`) will be initialized and used to protect the critical section, ensuring that `pthread_mutex_lock()` is called before accessing the shared resource and `pthread_mutex_unlock()` is called immediately after. The main thread will then wait for all worker threads to complete their execution using `pthread_join()`, after which the final, consistent value of the shared resource will be verified and displayed. The program's behaviour will also be tested and compared with and without mutex protection to clearly demonstrate the effect of synchronization on correctness.

5. Operating Systems Concepts / Linux APIs Used

(List the relevant Operating Systems concepts, Linux system calls, POSIX APIs, or mechanisms that will be used.)

OS Concept / Linux API / System Call	Purpose in the Project
Process & Thread Management	Creating and managing multiple threads within a single Linux process using the POSIX Threads (Pthreads) library.
<code>pthread_create()</code>	Used to create new threads that execute a specified function concurrently with the main thread.
<code>pthread_join()</code>	Used by the main thread to wait for the completion of worker threads before proceeding, ensuring proper synchronization of program flow.
Mutex (<code>pthread_mutex_t</code>)	Used to implement mutual exclusion, allowing only one thread at a time to access a shared/critical section of code.
<code>pthread_mutex_lock()</code> / <code>pthread_mutex_unlock()</code>	Used to lock and unlock the critical section around shared data, preventing race conditions during concurrent access.

Critical Section & Race Condition Handling	Demonstrates how uncontrolled concurrent access to shared memory causes inconsistent results, and how synchronization resolves it.
--	--

6. Individual Contribution

(Clearly specify the responsibility of each team member in the project.)

Roll Number	Student Name	Individual Responsibility
2520090170	S Svara Haasini	Designed the overall program structure, implemented thread creation using <code>pthread_create()</code> , and prepared the project documentation and report.
2520090227	B Sudharshini	Implemented the mutex locking mechanism (<code>pthread_mutex_lock/unlock</code>) to protect the critical section and tested the program for race conditions.
2520090190	Sumanjali	Implemented thread joining logic using <code>pthread_join()</code> , verified output consistency, and handled debugging and testing on the Linux environment.

7. Tools / Platforms / Software Used

(List the programming languages, operating system, libraries, tools, and software planned for the project.)

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system platform on which the application is developed, compiled, and executed.
C / C++	Programming language used to implement thread creation, mutex synchronization, and the overall application logic.
POSIX Threads (Pthreads) Library	Provides the API (<code>pthread_create</code> , <code>pthread_join</code> , <code>pthread_mutex_lock/unlock</code> , etc.) used to create and synchronize threads.
GCC Compiler	Used to compile the C source code into an executable, linked with the pthread library (<code>-lpthread</code>) on Linux.
VS Code / GDB	Used as the code editor and debugger for writing, testing, and debugging the multithreaded application.

8. Expected Outcome

(Describe the expected functionality, output, and outcome of the project.)

At the end of this project, a fully functional multithreaded Linux application will be developed that correctly creates, manages, and synchronizes multiple POSIX threads. The application will demonstrate safe concurrent access to shared resources, with mutex locks successfully preventing race conditions and ensuring that the final output is accurate and consistent across multiple runs.

The project will also produce a clear comparison showing inconsistent, unpredictable results when threads access shared data without synchronization, versus consistent, correct results when mutex-based synchronization is applied.



This outcome will provide practical, hands-on understanding of thread creation, critical sections, mutual exclusion, and safe resource sharing in a real Linux environment.

9. GitHub Repository

GitHub Repository Link: <https://github.com/haasinisalihundam-netizen/KLH-CSIT-2026-8-MULTITHREADED-LINUX-APPLICATION>

Repository Created: Yes

☒ Yes ☐ No

Faculty Name and Signature

Faculty Name: Mr. M. Raghupathi

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☒ Yes ☐ No

Date: _____

Faculty Signature: _____